

Verification Continuum™

Siloti® User Guide and Tutorial

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Audience	5
Customer Support	5
Synopsys Statement on Inclusivity and Diversity	6

1. Introduction	7
Overview	7
Technology Overview	8
Visibility Analysis Engine	8
Data Expansion Engine	8
Abstraction Correlation Module	9
Application Overview	9
Full Chip Simulation	9
Emulation and Hardware Acceleration	10

2. Launching Techniques	11
Reference Source Files on the Command Line	11
Compile Source Code into a Library	11
Reference Design and FSDB on the Command Line	12
Replay a File	12
Perform Behavior Analysis on the Command Line	13
Perform Data Expansion on the Command Line	13
Loading when Design and FSDB Hierarchies do not Match	14
Recommended Method for Performing Essential Signal Analysis	15

3. Application Tutorials	16
Prerequisites	16
Full Chip Simulation	17
Input Requirements	17
Determine Signals to Dump	17
Command Line	18

Contents

Setup for Data Expansion	19
Command Line	19
Debug With Data Expansion	20
Emulation	23
Input Requirements	23
Determine Signals to Dump	23
Command Line	24
Setup for Data Expansion	25
Command Line	25
Debug With Data Expansion	25
Debug Gate-Level Results on the RTL Design	28
Concept	28
4. Appendix A: Name Mapping Rules	29
Regular Expression Examples	30
Predefined Rules Examples	31
Applying Rules	32
Example 1:	32
Example 2:	33
Example 3:	33
5. Appendix B: Design Extraction Messages Overview	34

Preface

This book is designed to allow you to quickly become proficient with the Siloti system. The manual should be read from beginning to end, although you may skip any sections with which you are already familiar.

This chapter describes the following section:

- [Audience](#)
- [Customer Support](#)
- [Synopsys Statement on Inclusivity and Diversity](#)

Audience

The audience for this manual includes design, verification, and silicon validation engineers who require enhanced visibility into their full-chip simulation, emulation, or hardware acceleration results.

This document assumes that you have a basic knowledge of the platform on which your version of the Siloti system runs, UNIX or Linux, and that you are knowledgeable in Verilog or VHDL and digital logic design and verification.

Also, depending on the application, familiarity with the Verdi platform, softwarebased simulation, emulation, or hardware acceleration concepts is helpful.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetsynopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Contents

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/ permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customerfacing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction

In this chapter:

- [Overview](#)
- [Technology Overview](#)
- [Application Overview](#)

Overview

The *Siloti™ Visibility Automation System* speeds up simulation by eliminating the overhead associated with recording signal values and by enabling full visibility of design activity based on a relatively small amount of signal data captured during full-chip simulation, regression simulation, and hardware emulation/acceleration. The Siloti system, which is integrated with the *Verdi® Automated Debug System*, makes verification more efficient by enabling comprehension of complex IC and system-on-chip (SoC) behavior in conjunction with simulation and hardware-based verification applications. Using this capability, you can

- Achieve full visibility into the functional operation of designs with limited impact on verification performance
- Enable analysis and debug of gate-level verification results on the RTL design
- Reduce the overall cost of verification

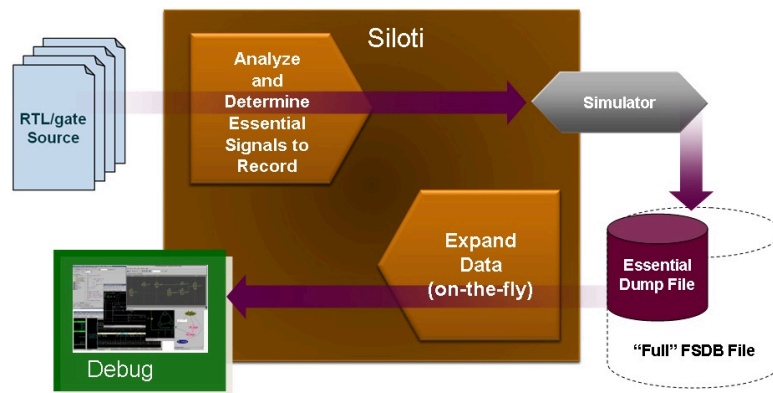
The Siloti system mitigates the expense of obtaining enough signal value information to debug complex problems found during emulation, acceleration, and full-chip simulation. Since the Siloti system automatically analyzes a design to determine the essential signals (signals whose values must be captured during verification), infers values for signals not captured, and maps gate-level signal data back to the RTL; design comprehension and debug is greatly enhanced and simulation time is dramatically reduced.

The complementary Verdi platform is an advanced solution for debugging digital designs that increases design productivity with complex System-on-Chip (SoC), ASIC, and FPGA designs. The powerful technology provided in the Verdi platform helps you comprehend complex and unfamiliar design behavior, automate difficult and tedious debug processes, and unify diverse and complicated design environments. The Verdi platform and the

additional *nAnalyzer* and *nECO* modules enable engineers to locate, isolate, understand, and resolve bugs in a fraction of the time of traditional solutions.

Technology Overview

Figure 1 Siloti Visibility Enhancement Products Overview



The Siloti system provides the engines and modules mentioned below.

Visibility Analysis Engine

The Visibility Analysis engine analyzes an RTL description or a gate-level netlist to determine the minimum set of signals needed to provide full visibility into a designated portion of the design. The data for these essential signals provides the input for the processing performed by the Data Expansion engine described below. Since the set of essential signals is typically much smaller than the full set of signals, capturing data for only the essential signals significantly improves simulation performance.

To determine the essential signals, the Visibility Analysis engine relies on a complete set of logic equations inferred from the HDL description. For each functional block, it recursively traverses the inferred logic, extracts the primary inputs, register outputs, memory outputs, and the outputs of non-synthesizable blocks, then writes these signals into a file that can be used to control dumping by simulators or other verification tools.

Data Expansion Engine

Understanding abnormal device behavior is difficult when only a subset of data is available from the device. The Data Expansion engine is applied to compute missing signal data 'on-demand' using knowledge of the design function. After the values of the missing signals are computed, debugging techniques that rely on signal information over time can be utilized to allow tracing of causes from effects. When used in conjunction with

the Visibility Analysis engine, the Data Expansion engine maximizes visibility into the operation of the design during debug.

Given the values of registers and primary inputs, the Data Expansion engine sorts the combinational logic between the registers using a linear ordering algorithm and computes the values for these signals using a cycle-based evaluation engine (that is, logic and wire delays are not considered). The computed values are sent through a data agent to the application window or windows in which the user is exploring the design.

The Data Expansion engine is unique in that the values of signals are computed “on the fly.” In other words, only the values of the signals that are required by the current debug operation or display are computed. To do this, a new value called “NC” (Not Computed) is introduced in the FSDB file and windows that display signal data. When the value of a signal has not been captured, and has not been expanded by the Data Expansion engine, the value of NC is stored and displayed. After expansion, the signal values are shown and a different color is used to differentiate expanded values from signal data captured during verification.

Abstraction Correlation Module

A gate-level netlist is typically hard to understand when generated by synthesis tools. To easily understand the meaning of gate-level logic, it is important to have an ability to correlate gate-level verification results back to the RTL design descriptions. The Abstraction Correlation module can be applied in the background to map gate-level signal data to the RTL design for easier debug and understanding.

The Abstraction Correlation module interoperates seamlessly with the Data Expansion engine to automatically enable analysis and debug of the gate-level results with full visibility on the RTL design.

Application Overview

The Siloti system supports the following applications:

Full Chip Simulation

Full-chip simulation often requires a trade-off between performance and debug. Generally, a greater understanding of the behavior, and therefore quicker debug, is achieved by dumping as much signal data as possible. Unfortunately, a high amount of dumping significantly degrades simulation performance. As a result, many verification teams avoid signal dumping in favor of iterating lengthy simulations with signal dumping only after an error is discovered. The number of total iterations using this approach is not possible to predict and can significantly impact the design schedule.

By using a methodology that limits the dumping to a small, but critical subset of signals, verification teams can achieve full-chip functional simulation debug with minimal performance impact. This methodology relies on the selection of a minimal number of dumped signals, yet provides visibility of most signal values. The Visibility Analysis engine assists with the selection of signals to be included in the dump list to ensure that the module signals can be observed.

Prior to running simulation, the simulator is set to only dump the identified set of essential signals. After the simulation, the dumped data is processed with the Data Expansion engine. The Data Expansion engine operates on the available data to compute the missing signal data in cycle-accurate fashion, without consideration of timing delays. This expanded data can then be processed by the Siloti system and analyzed using advanced debug techniques as if all signals were captured during simulation.

Emulation and Hardware Acceleration

Some emulation and hardware acceleration systems present significant challenges for debug due to the limited observability of internal signal data and the unfamiliar gate-level representation of the design. Because these verification systems are hardware-based, engineering teams must run multiple iterations of synthesis and compilation steps to configure or insert debug logic, such as, internal logic analyzers, and then re-execute the tests. Setup of emulation systems for register dumping impacts performance and turn-around time. Although dumping registers or inserting logic analyzers allows observation of user-selected signals, in many cases these techniques fail to include the signals required for debugging a specific problem. As a result, an unpredictable number of iterations is possible, and still may not provide visibility of all signals at the desired RTL view.

The combination of the Data Expansion engine and the Abstraction Correlation module, working transparently on a limited amount of signal data, enables debug automation and makes it possible to trace problems on the RTL design. The Abstraction Correlation module maps gate-level signal data captured from the registers to the corresponding RTL registers. The mapped data is then processed with the Data Expansion engine to make RTL combinational values visible. The combined outcome is RTL-centric debug with high observability of most internal signals of the emulated design. An optional step to improve the emulation performance is to use the Visibility Analysis engine to reduce the number of registers dumped during each run. With appropriate emulator interfaces, this step can be user controlled using the emulation setup.

2

Launching Techniques

This chapter summarizes the various methods for starting the Siloti system, loading the design, and loading the simulation results stored in the Fast Signal Database (FSDB).

- [Reference Source Files on the Command Line](#)
- [Compile Source Code into a Library](#)
- [Reference Design and FSDB on the Command Line](#)
- [Replay a File](#)
- [Perform Behavior Analysis on the Command Line](#)
- [Perform Data Expansion on the Command Line](#)
- [Loading when Design and FSDB Hierarchies do not Match](#)
- [Recommended Method for Performing Essential Signal Analysis](#)

Reference Source Files on the Command Line

This method loads the design directly from the source files. It can only be used for single-language designs.

1. Use the following command to reference the source files on the command line:

```
% verdi -f <source_file_name>
```

where `source_file_name` is a file that lists all of the HDL source files.

2. Then use the **File -> Open Waveform File** command to load the FSDB.

Compile Source Code into a Library

This method must be used if you have a mixed-language design.

1. Use the utility program *vhdlcom* (supplied with the Verdi and Siloti installation) to compile your VHDL source code into a library and use *vericom* for Verilog code:

```
% vericom -lib <libName> block1.v block2.v ...  
% vhdlcom -lib <libName> block1.vhd block2.vhd ...
```

2. Then use the following command to load the compiled design:

```
% verdi -lib <libName> -top <TopBlock>
```

where *libName* is the compiled library and *TopBlock* is the highest-level block you wish to see.

Reference Design and FSDB on the Command Line

1. Use the following command to reference both the source files and the FSDB on the command line:

```
% verdi -f <source_file_name> -ssf <fsdb_file_name>
```

where *source_file_name* is the source file name and *fsdb_file_name* is the name of the FSDB file.

2. Use the following command to reference both the compiled library and the FSDB on the command line:

```
% verdi -lib <libName> -top <TopBlock>  
-ssf <fsdb_file_name>
```

where *libName* is the compiled library and *TopBlock* is the highest-level block you wish to see and *fsdb_file_name* is the name of the FSDB file.

Note:

If the specified FSDB file is an Essential Signal FSDB, you are presented with a *Question* dialog related to Data Expansion. If you plan to use the Siloti system, click **Yes**; otherwise, click **No**. Data Expansion can always be started or the options changed by invoking the **Tools -> Visibility -> Setup Data Expansion** command.

Replay a File

Use the following command to replay a file containing commands that loads both the design and the FSDB (and perform a variety of other tasks, if desired):

```
% verdi -play <command_file_name>
```

where *command_file_name* is a file with a number of Tcl commands.

Perform Behavior Analysis on the Command Line

To perform Behavior Analysis on the command line, you would specify the **-ba** option.

1. Referencing the source files:

```
% verdi -f <source_file_name> -ba -ba_mode <mode> -ssf  
<fsdb_file_name>
```

where `source_file_name` is the source file name and `fsdb_file_name` is the name of the FSDB file.

2. Referencing the compiled library:

```
% verdi -lib <libName> -top <TopBlock> -ba  
-ba_mode <mode> -ssf <fsdb_file_name>
```

where `libName` is the compiled library and `TopBlock` is the highest-level block you wish to see and `fsdb_file_name` is the name of the FSDB file.

Note:

The `-ba` option executes Behavior Analysis using the Behavior Analysis settings from the *novas.rcresource* file unless you specifically include them on the command line. Refer to the *verdi* utility description in the *Verdi and Siloti Command Reference* for a list of Behavior Analysis options.

Note:

The `-ba_mode` option specifies the default mode for Behavior Analysis. The mode options are Working Scope Behavior Analysis (wsba) or Module Based Behavior Analysis (mbba). The mode can also be specified by environment variable, refer to the *Behavior Analysis* section of *Appendix A* in the *Verdi and Siloti Command Reference* for details.

Perform Data Expansion on the Command Line

To perform Data Expansion on the command line, you would specify the `-de` or `-DE` option in addition to loading the FSDB file and performing Behavior Analysis. The `-de` option performs Data Expansion in auto time window mode. The `-DE` option performs Data Expansion based on the specified options.

1. Referencing the source files:

```
% verdi -f <source_file_name> -ba -ba_mode <mode> -de -ssf  
<fsdb_file_name>
```

or

```
% verdi -f <source_file_name> -ba -ba_mode <mode> -DE "-start_time 0  
-end_time 1000" -ssf <fsdb_file_name>
```

where `source_file_name` is the source file name and `fsdb_file_name` is the name of the FSDB file.

Note:

`-f <source_file_name>` option can be replaced with `-lib <library_name> -top <TopBlock>` option if your design is precompiled to a library.

Note:

The `-ba` option is optional if either the `-de` or `-DE` option is specified.

Note:

The `-ba_mode` option specifies the default mode for Behavior Analysis. The mode options are Working Scope Behavior Analysis (wsba) or Module Based Behavior Analysis (mbba). The mode can also be specified by environment variable, refer to the *Behavior Analysis* section of *Appendix A* in the *Verdi and Siloti Command Reference* for details.

Note:

The `-ba/-de` option executes Behavior Analysis using the Behavior Analysis settings from the *novas.rc* resource file unless you specifically include them on the command line. Refer to the *verdi* utility description in the *Verdi and Siloti Command Reference* for a list of Behavior Analysis options.

Loading when Design and FSDB Hierarchies do not Match

Use the following command if you dumped the FSDB file for the entire design but you only want to load a portion of the design for debug.

```
% verdi -f <source_file_name> -ssf <fsdb_file_name> -vtop <map_file_name>
```

where `source_file_name` is the source file name, `fsdb_file_name` is the name of the FSDB file and `map_file_name` is the name of the map file.

The map file is used to match the design hierarchy to the hierarchy in the FSDB file so that the simulation results are correctly annotated on the source code and schematic views. The Siloti system automatically generates a virtual hierarchy in the design according to the definitions in the `.map` file. The syntax is as follows:

```
module_name = hierarchical_instance_path
```

Note:

The map file matches the case sensitivity of the associated language. For example, Verilog is case sensitive so if the module definition is all capitalized, the map file description needs to be as well (that is, `cpu` does not equal `CPU`). VHDL is not case sensitive (that is, `cpu` equals `CPU`).

If you incorrectly enter the module name in the map file, you may see an error similar to the following in the compiler.log (**File -> View Import Log**):

```
*Error* view cpu is not defined for inst i_cpu
"virtual_top_autov_15123.gen:", 7:
```

This error needs to be eliminated. Check the module definition in the source code and confirm that the map file matches it exactly.

Recommended Method for Performing Essential Signal Analysis

The recommended method for performing Essential Signal Analysis is to use the `esa` utility.

1. Referencing the source files:

```
% esa -f <source_file_name> -db <ESDB_file_name> -bas <top_scope>
```

where `source_file_name` is the source file name, `top_scope` is the design top scope for Behavior Analysis, and `ESDB_file_name` is the file name for the generated essential signal database.

2. Referencing the compiled library:

```
% esa -lib <libName> -db <ESDB_file_name> -top <TopBlock> -bas
<top_scope>
```

where `libName` is the compiled library, `TopBlock` is the highest-level block you wish to see, `top_scope` is the design top scope for Behavior Analysis, and `ESDB_file_name` is the file name for the generated essential signal database.

Note:

If `-bas` is not specified, the working scope is obtained from the `novas.rc` resource file (if one exists); otherwise, the default is the top module of the design.

Note:

If you are using a gate-level design, the `-cellModel 0` option must also be included to enable the symbol library.

3

Application Tutorials

This chapter introduces you to applications where you can use the Siloti system and familiarizes you with its core concepts and use models.

This chapter consists of the following sections:

- [Prerequisites](#)
- [Full Chip Simulation](#)
- [Emulation](#)
- [Debug Gate-Level Results on the RTL Design](#)

Prerequisites

Before you begin the tutorials, you (or your system administrator) must install Verdi and Siloti systems as described in the *Installation and System Administration Guide*. You must also complete the following steps to set up the Siloti environment and the files required for tutorials:

Note:

The optional `Novas-2012??-demo.tar.gz` package must be installed (where 2012 corresponds to the year and ?? corresponds to the month).

1. Add the Verdi application (binary) to the search path and specify the search path to the license file.

For details, see the *Setting Up the Environment and Running the Software* section in the *Installation and System Administration Guide*.

2. Create a working directory using the following command:

```
% mkdir <working_dir>
```

3. All of the tutorial data resides in the `$VERDI_HOME/demo` directory. Make a copy of these demo files in your working directory using the following directory:

```
% cp -r <VERDI_INST_DIR>/demo <working_dir>
```

Full Chip Simulation

Many companies avoid dumping all data during their full-chip simulations due to its time-related cost of dumping entire simulation data during large simulation runs. A methodology is typically employed in which problems are detected by the verification environment – testbench, assertions, etc – and a second simulation run is required with dumping turned “on”. In some cases, this can cost an extra day of turnaround time – every time a problem is detected during simulation – waiting for the second simulation to complete before debug can begin. The Siloti system provides significant benefits in the following two specific areas:

1. **Visibility Analysis** – Determines the optimal (minimal but sufficient) set of signals that is captured during simulation to enable full visibility into design operation during debug using the Data Expansion engine.
2. **Data Expansion** – Processes the limited data captured during simulation and calculates or expands the values for the combinational signals to enable full visibility into design operation and thus, full and efficient debug.

This technology can enhance the typical full-chip simulation methodology. It retains full visibility into design operation and access to automated debug techniques while capturing only a minimal amount of signal data during simulation. This eliminates the need for multiple simulations before debug can begin.

This section consists of the following subsections:

- [Input Requirements](#)
- [Determine Signals to Dump](#)
- [Setup for Data Expansion](#)
- [Debug With Data Expansion](#)

Input Requirements

To use the Siloti system with full-chip simulation, the following items are required:

- RTL design or gate-level design with symbol libraries to generate the essential signal list.
- FSDB or VCD file with simulation results for the essential signals.

Determine Signals to Dump

The Visibility Analysis engine answers the question: Which signals are captured by the full-chip simulation to guarantee full visibility into the critical portions of the design? Given

a set of signals, a block, or the full design that you want to debug, you want to determine the essential set of signals that needs to be passed to the simulator to capture during simulation.

Consider the scenario in which you want to dump the minimal set of signals during simulation. It still allows for full visibility into design operation after the fact via the Data Expansion engine. You can generate the essential signal list using the command line.

Note:

The Siloti version that generates the essential signal list and that expands the essential signal FSDB file must be same.

Command Line

To perform Visibility Analysis, perform the following steps from the command line:

1. Change to the RTL directory using the following command:

```
% cd <working_dir>/demo/siloti/rtl
```

2. In a shell window, execute the following command to perform Essential Signal Analysis:

```
% esaesa -f run.f -db es -bas tb_CPUsystem
```

Note:

- 1) If `-bas` is not specified, the working scope is obtained from the `novas.rc` resource file (if one exists). Otherwise, the default is the top module of the design.
- 2) The `-db` option specifies the file name of the essential signal database. A directory `<ESDB_filename>.esdb++` is created for storing the essential signal database.
- 3) If you are using a gate-level design, include the `-cellModel 0` switch to enable the symbol library.

For details on the `esa` utility, see the *Utilities* chapter of the *Verdi and Siloti Command Reference*.

The essential signal database is saved in the `es.esdb++` directory, that is used with the `+fsdb+esdb="ESDB_filename"` command-line option during simulation. For details, see the *FSDB Dumping Commands* chapter of the *Linking Novas Files with Simulators and Enabling FSDB Dumping* manual. You need to link the appropriate Novas object file for FSDB dumping to your simulator. For details, see the *Linking Novas Files with Simulators to Enable FSDB Dumping* document.

Setup for Data Expansion

Assume that the RTL description provides the input for the Visibility Analysis engine, which was used to generate the list of essential signals. This file was then used as input to the `fsdbDumpvarsToFile` dump command during the full-chip simulation of the RTL design to generate the FSDB file. For details on the dumping commands, see the *FSDB Dumping Commands* chapter of the *Linking Novas Files with Simulators and Enabling FSDB Dumping* manual.

You can setup Data Expansion using the command line.

Note:

The Siloti version that generates the essential signal list and that expands the essential signal FSDB file must be same.

Command Line

1. Change to the RTL directory using the following command:

```
% cd <working_dir>/demo/siloti/rtl
```

2. Execute the following command to load the design and FSDB file, perform Behavior Analysis, setup Data Expansion in the auto-time window mode, and display all design signals (in the FSDB file and those that are expanded).

```
% verdi -f run.f -ssf ess.fsdb -ba -ba_mode mbba -de &
```

Note:

- 1) The `-ba` switch is optional if either the `-de` or `-DE` switch is specified.
- 2) The `-ba/-de` switches use the Behavior Analysis settings from the `novas.rc` resource file unless you specifically include them on the command line. For a list of Behavior Analysis options, see the *verdi* utility description in the *Verdi and Siloti Command Reference*.
- 3) The `-ba_mode` switch specifies the default mode for Behavior Analysis. The suggested mode for Data Expansion is Module Based Behavior Analysis (MBBA). The mode is also specified by an environment variable, see the *Behavior Analysis* section of *Appendix A* in the *Verdi and Siloti Command Reference* for details.
- 4) You can use `-DE "options"` in place of `-de` to setup Data Expansion with the specified options. For a list of `-DE` options, see the *verdi* utility description in the *Verdi and Siloti Command Reference*.

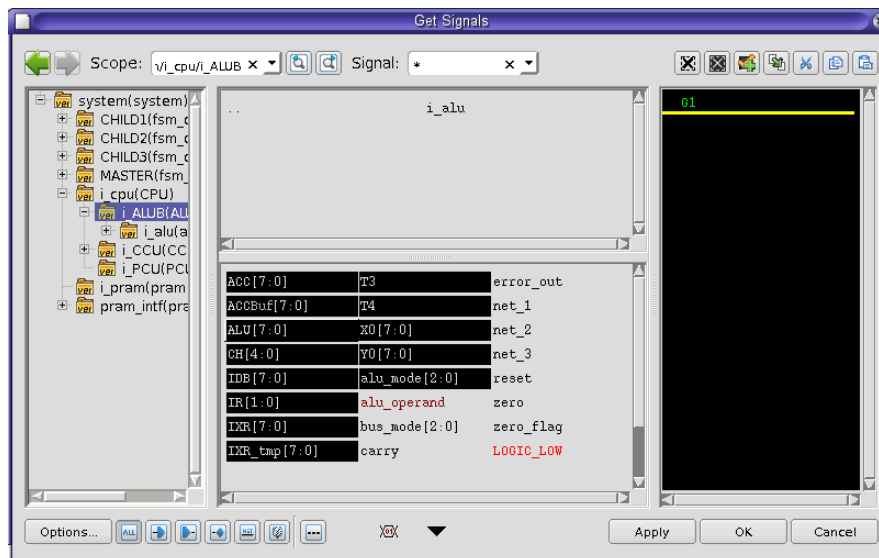
Debug With Data Expansion

The limited data dumped during simulation using the Visibility Analysis process described above is useful, but it does not provide the visibility necessary for efficient debug. The Data Expansion engine processes the limited data to enable full visibility into design operation. Thus, efficient understanding and debug of full-chip simulation results in a minimal impact on simulation runtime.

Consider a scenario in which you apply the Data Expansion engine and debug an unexpected transition of **3 -> aa** on `AluBuf[7:0]` at Time 826.

1.  In the main window, click the **New Waveform** toolbar icon.
2.  In the *nWave* frame, click the **Get Signals** icon or select the **Signal -> Get Signals** command.
3. In the *Get Signals* hierarchy browser pane, click the + icon to expand the **i_CPUsystem** scope and then the **i_CPU** scope.
4. Click to select **i_ALUB**. Note that some of the signals are displayed in the bold style - these are signals that are not dumped to FSDB and are expanded (calculated) by the Data Expansion engine.
5. Drag-left to select several signals; make sure to include `AluBuf[7:0]` in the selection.

Figure 2 Get Signals With Expanded Signals

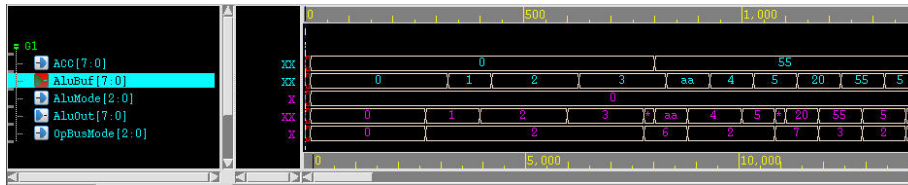


6. Click the **OK** button to add the signals to the waveform.

Note:

You can alternatively select a scope in the design browser window and use the middle-mouse button to drag and drop the selected scope to the *nWave* window. The signals are automatically calculated. For example, in the design browser window, double-click to expand the **i_CPUsystem** scope and then, the **i_CPU** scope. Left-click to select **i_ALUB** and use the middle-mouse button to drag and drop the selected scope to the *nWave* window.

Figure 3 *nWave With Expanded Signals*



7. Double-click the **3 -> aa** transition on `AluBuf[7:0]` at Time 826 in *nWave* to locate the active driver in the *Source Code* frame.
8. In the main window, select the **Source -> Active Annotation** command (or press x on the keyboard) so you can see the values overlaid on the Source code.

The values for all signals are displayed in the annotation. The expanded signals appear in a different color (purple) to indicate that they are derived and do not come from the FSDB file, similar to the following example:

Figure 4 *Active Annotation With Expanded Data*

```

80 alu_i_alu (.a(aluInA) , .b(aluInB) , .cin(CarryMode) , .select(AluMode)
81   , .out(AluOut) , .carry(carryOut) , .zero(isZero));
82
83 always @(posedge CLOCK4 or negedge RESET)
84 begin
85     if (!RESET)
86         AluBuf = 0;
87     else
88         AluBuf = #1 AluOut;
89 end

```

Note:

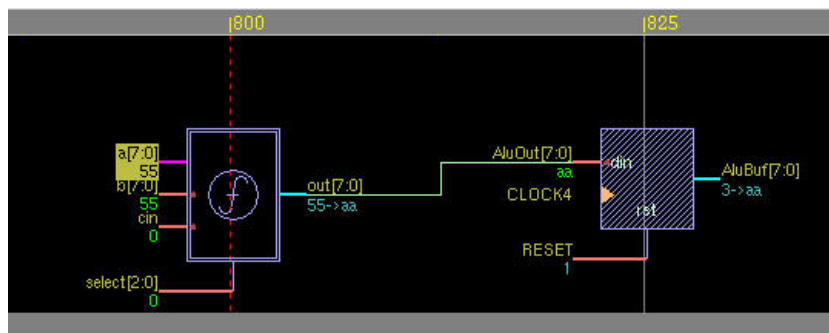
The expanded signal color for Active Annotation is changed with the **ComputedSignal** key in the **Type** field on the **Color** page in the **Source Code** folder of the *Preferences* form (invoked via the **Tools -> Preferences** command).

9. Click **AluOut**. Invoke the right-mouse button context menu and select the **Active Trace** command (or press **Ctrl+t** on the keyboard) to trace the active driver in the source code.

The source code changes to a case statement. Even with the signal values and annotation, it is difficult to continue debugging from here as it is not obvious which signal to trace. However, now that all of the signals values are available, advanced debug capabilities are used.

10. In the *nWave* window, put the cursor on the **3 -> aa** transition on **AluBuf[7:0]** at Time 826.
11. Right-click to invoke the right-mouse button context menu and select the **Temporal Flow View -> New Temporal Flow View** command to open the *Temporal Flow View* window.
12. In the *Temporal Flow View* window, double-click the **AluOut[7:0]** input to expand one level.
13. Click to select the **a[7:0]** input on the function symbol, similar to the following example:

Figure 5 Tracing in the Temporal Flow View Window



14. Right-click to invoke the right-mouse button context menu, select the **Trace This Value** command to automatically trace the 55 value over multiple cycles.

The Data Expansion engine processes the limited data captured during full-chip simulation and calculates the values for combinational signals to enhance visibility into design operation and enable automated debug capabilities.

Note:

For big gate-level designs, it is slow to use the *Temporal Flow View* together with Data Expansion.

Emulation

Many companies today augment their verification strategy with low-cost commercial emulation solutions – either for acceleration or firmware/software development purposes. In many cases, these systems provide minimal observability of internal signal data. It is often limited to some subset of registers in the design, which has a significant adverse impact on understanding and debugging problems that occur during emulation. The Siloti system can provide significant benefits in the following three specific areas:

1. **Visibility Analysis** – Determines the optimal (minimal but sufficient) set of signals that the emulator must capture to guarantee full visibility into the critical portions of the design and thus, efficient debug.
2. **Data Expansion** – Processes the limited data captured by the emulator and calculates or expands the values for the combinational signals to enhance visibility into design operation and to enable full and efficient debug.
3. **Correlation** – Maps the error location or emulation data back to the RTL design to correlate it for efficient analysis and debug.

This chapter consists of the following sections:

- [Input Requirements](#)
- [Determine Signals to Dump](#)
- [Setup for Data Expansion](#)
- [Debug With Data Expansion](#)

Input Requirements

To use the Siloti system with emulation, you require the following items:

- RTL design or gate-level design with symbol libraries to generate the essential signal list.
- FSDB or VCD file with emulation results for the essential signals.

Determine Signals to Dump

Use the Visibility Analysis engine to determine which signals to dump. Visibility Analysis answers the question: Which signals are captured by the emulation system to guarantee full visibility into the critical portions of the design? Given a set of signals or a block for which you must have full visibility after the fact, the goal is to determine the essential set of signals that is captured by the emulator system for later processing using the Data Expansion engine.

Note:

Some tools have their own way of choosing critical signals (for example, registers). Therefore, it may be appropriate to start with that mechanism and skip this step.

Assume you want to debug the entire chip after you run emulation and that the RTL description generates the essential signal list and provides the input for the emulator. The results for the essential signals are then captured during emulation and match the signal names used in the RTL description.

Note:

If the results from emulation match the signal names used in the netlist but you want to debug on the RTL design, see the [Debug Gate-Level Results on the RTL Design](#) section.

You can generate the essential signal list using the command line.

Note:

The Siloti version that generates the essential signal list and that expands the essential signal FSDB file must be the same.

Command Line

Essential Signal Analysis is also performed on the command line using `esa`. The equivalent command executed in a shell window is as follows:

```
% esaesa -f run.f -db es -bas tb_CPUsystem
```

Note:

- 1) If `-bas` is not specified, the working scope is obtained from the `novas.rc` resource file (if one exists). Otherwise, the default is the top module of the design.
- 2) If you are using a gate-level design, the `-cellModel 0` option is also included to enable the symbol library.

For details on the `esa` utility, see the *Utilities* chapter of the *Verdi and Siloti Command Reference*.

To convert the essential signal database to a text file to be an input of the emulator prototype, use the `esdb2txt` utility. In a console, type the following:

```
% esdb2txt es -plain_eslist es.list &
```

Using the Visibility Analysis engine, you can minimize the set of signals captured by the emulator to guarantee full visibility into critical design operation after the fact using the Data Expansion engine.

Setup for Data Expansion

Assume the Visibility Analysis engine was used to generate the list of all essential signals. This file was then used as an input to the emulator/prototype system to generate an FSDB file containing data for the essential signals and the results match the RTL design.

Note:

The Siloti version that generates the essential signal list and expands the essential signal FSDB file must be the same.

Command Line

1. Change to the RTL directory using the following command:

```
% cd <working_dir>/demo/siloti/rtl
```

2. To load the design and FSDB file, execute the command below. Also, perform Behavior Analysis, setup Data Expansion in the auto-time window mode, and display all design signals (in the FSDB file and those that are expanded).

```
% verdi -f run.f -ba -ba_mode mbba -ssf ess.fsdb -de &
```

Note:

- 1) The `-ba` switch is optional if either the `-de` or `-DE` switch is specified.
- 2) The `-ba/-de` switches use the Behavior Analysis settings from the `novas.rc` resource file unless you specifically include them on the command line. For a list of Behavior Analysis options, see the *verdi* utility description in the *Verdi and Siloti Command Reference*.
- 3) The `-ba_mode` switch specifies the default mode for Behavior Analysis. The suggested mode for Data Expansion is Module Based Behavior Analysis (MBBA). The mode is also specified by the environment variable, refer to the *Behavior Analysis* section of *Appendix A* in the *Verdi and Siloti Command Reference* for details.
- 4) Use `-DE "options"` in place of `-de` to setup Data Expansion with the specified options. For a list of `-DE` options, see the *verdi* utility description in the *Verdi and Siloti Command Reference*.

Debug With Data Expansion

The limited data available from many emulation systems can hinder the ability to understand and debug system operation when problems are detected. The Data Expansion engine processes limited data to enhance visibility into critical design operation

and thus, enable efficient and automated debug of problems detected while running emulation.

1.  In the main window, click the **New Waveform** toolbar icon.
2.  In the *nWave* window, click the **Get Signals** toolbar icon or select the **Signal -> Get Signals** command.
3. In the *Get Signals* hierarchy browser pane, click the + icon to expand the **i_CPUsystem** scope and then, the **i_CPU** scope.
4. Left-click to select **i_ALUB**. Note that some of the signals are displayed in the bold style - these are signals that were not dumped to the FSDB and are expanded (calculated) by the Data Expansion engine.
5. Drag-left to select several signals; make sure to include **AluBuf[7:0]** in the selection.
6. Click the **OK** button to add the signals to the waveform.

Note:

Alternatively, you can select a scope in the design browser window and use the middle-mouse button to drag and drop the selected scope to the *nWave* window. The signals are automatically calculated. For example, in the design browser window, double-click to expand the **i_CPUsystem** scope and then the **i_CPU** scope. Left-click to select **i_ALUB** and use the middle-mouse button to drag and drop the selected scope to the *nWave* window.

7. Double-click on the **3 -> aa** transition on **AluBuf[7:0]** in *nWave* to locate the active driver in the *nTrace* source code pane.
8. In the main window, select the **Source -> Active Annotation** command (or press x on the keyboard) so you can see the values overlaid on the source code.

The values for all signals are now displayed in the annotation and the expanded signals appear in a different color (purple) to indicate that they are calculated and do not come from the FSDB file, similar to the following example:

Figure 6 Active Annotation with Expanded Data

```

80 alu i_alu (.a(aluInA) , .b(aluInB) , .cin(CarryMode) , .select(AluMode)
      , .out(AluOut) , .carry(carryOut) , .zero(isZero));
81
82
83 always @(posedge CLOCK4 or negedge RESET)
84 begin
85     if (!RESET)
86         AluBuf = 0;
87     else
88         AluBuf = #1 AluOut;
89 end

```

Note:

To change the expanded signal color for Active Annotation, use the **ComputedSignal** key in the **Type** field on the **Color** page under the **Source Code** folder of the *Preferences* form (invoked via the **Tools -> Preferences** command).

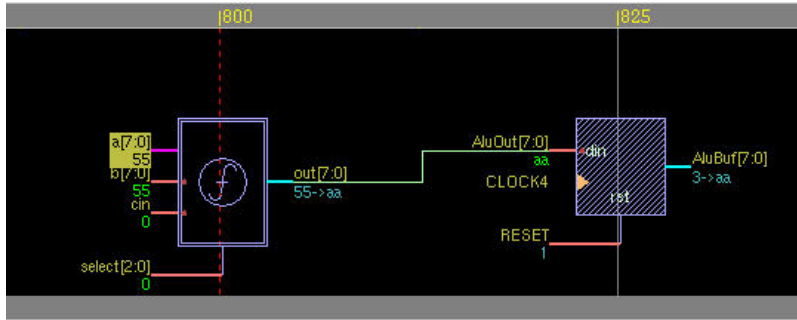
- Click to select **AluOut**, invoke the right-mouse button context menu, and select the **Active Trace** command (or press **Ctrl+t** on the keyboard) to trace the active driver in the source code.

The source code changes to a case statement. Even with the signal values and annotation, it is difficult to continue debugging from here as it is not obvious which signal to trace. However, now that all of the signals values are available, advanced debug capabilities are used.

- In the *nWave* window, put the cursor on the **3 -> aa** transition on **AluBuf[7:0]** at Time 826.
- Right-click to invoke the right-mouse button context menu and select the **Create Temporal Flow View** command to open the *Temporal Flow View* frame.
- In the *Temporal Flow View* frame, double-click the **AluOut[7:0]** input to expand one level.

13. Left-click to select the **a[7:0]** input on the function symbol, similar to the following example:

Figure 7 Tracing in the Temporal Flow View Frame



14. Right-click to invoke the right-mouse button context menu and select the **Trace This Value** command to automatically trace the 55 value over multiple cycles.

The Data Expansion engine processes the limited data captured from the emulator and calculates the values for combinational signals to enhance visibility into design operation and to enable automated debug capabilities.

Note:

If the results from the emulator match the netlist but you want to debug on the RTL design, see the [Debug Gate-Level Results on the RTL Design](#) section.

Debug Gate-Level Results on the RTL Design

Concept

This application allows the results to be mapped back to the RTL design and expanded to enable debug on the original RTL design. Please refer to the *Debug Gate-Level Results on the RTL Design* section in the *SilotiCorrelation.pdf* for the detailed usage.

4

Appendix A: Name Mapping Rules

The appendix describes the syntax for name mapping rules. You can access the *Perform CRDB Correlation* form by invoking **Correlation -> Gate/RTL Correlation -> Advanced Commands -> Perform CRDB Correlation** from the main Verdi and Siloti process (with the RTL design loaded). Refer to the *Perform CRDB Correlation* description in the *Visibility* chapter of the *Verdi and Siloti Command Reference* for complete details.

Figure 8 *Perform CRDB Correlation Form - Reg Exp Tab*

The screenshot shows the 'Prepare CRDB' dialog box with the 'Reg Exp Tab' selected. The dialog has four tabs: 'Default Rules', 'Hierarchy Change Rules', 'Module Specific Rules', and 'Scan Cell Rules'. The 'Regular Expressions' section contains two text input fields: 'Gate Matching Exp:' and 'RTL Substitution String:'. Below these fields are three buttons: 'Add', 'Insert', and 'Replace'. A large empty list box is positioned below the buttons, with 'Up', 'Down', 'Delete', and 'Delete All' buttons to its right. The 'Pre-defined Rules' section contains three text input fields: 'Hierarchical Separator:' (containing '/'), 'Tri-state:' (containing '%s_tri'), and 'Register:' (containing a complex regex pattern). At the bottom of the dialog are four buttons: 'Load...', 'Save...', 'Finish', and 'Exit'.

Select the **Default Rules** tab to specify name mapping rules for your design.

In this chapter:

- [Regular Expression Examples](#)
- [Predefined Rules Examples](#)
- [Applying Rules](#)

Regular Expression Examples

Using the **Regular Expression** section on the **Default Rules** tab in the *Perform CRDB Correlation* form, the following rules result in the described substitutions.

Figure 9 Regular Expression Examples

Regular Expressions

Gate Matching Exp:

RTL Substitution String:

u%d => m@1
abc_([A-Z]+)_%d => cba_@2_@1
^top => tb.newTop
pre_ =>
u1 => m2

1. Matching string is “u1” and substitution string is “m2”

Gate Matching Exp field contains: u1

RTL Substitution String field contains: m2

Causes strings like “top.u1.s1” to become “top.m2.s1”.

2. Match on the first %d and replace with @1

Rule: u%d => m@1

Causes “u1” to be mapped to “m1” and “u5” to be mapped to “m5”.

3. Match on string combination, then replace and change order.

Rule: abc_([A-Z]+)_%d => cba_@2_@1

Causes strings like “abc_PQR_555” to be mapped to “cba_555_PQR”.

4. If your gate-level design top does not match your RTL, you can use this rule to create a 'virtual top'.

Rule: ^top => tb.newTop

Causes strings like "top.u1.s1" to be mapped to "tb.newTop.u1.s1".

5. Delete the string by leaving the substitution string 'empty'.

Rule: pre_ =>

Causes strings like "top.pre_u1.s1" to be mapped to "top.u1.s1".

Note:

Regular Expressions work on a single design name only; they do not work for names crossing multiple hierarchies with delimiters. For example, assuming a full name of u1.u2.\escaped_name.abc.dfg.q, the rule u2.%s.q => u2.q does not work since the left hand side crosses multiple hierarchies but the rule abc.dfg => xyz works since the left hand side belongs to a single name.

Predefined Rules Examples

The **Predefined Rules** on the **Default Rules** tab of the *Perform CRDB Correlation* form follows the regular expression name matching and substitution string syntax; however, the data types are specially treated. The predefined rules support three gate-level to RTL correlation types: hierarchical separator, tri-state devices and registers.

Figure 10 *Perform CRDB Correlation Form - Predefined Rules*

Pre-defined Rules

Hierarchical Separator: /

Tri-state: %s_tri

Register: | %s_reg_%d_%d | %s_reg_%d__%d_ | %s_reg[%d][%d][%d] | %s_reg

1. **Hierarchical Separator** is the separator for flattened signal/instance. Only one character is allowed.

Example 1:

RTL uses: top.u1.u2.sig[0]

Gate uses: top.\u1!u2!sig[0]

Specify the hierarchical separator as '!'.

Example 2:

RTL uses: top.u1.u2.s1

Gate uses: top.u1#u2#s1

Specify the hierarchical separator as '#'.

Note:

Common hierarchy separators such as underscore '_' and dot '.' do not need to be specified as they are handled internally.

2. **Tri-state** is the format for tri-state instance and tri-state output net. Example:

Gate uses: bus1_tri

Specify the rule as: %s_tri

Causes the gate "bus1_tri" signal to be mapped to "bus1" in the RTL.

3. **Register** is the format for register instance and register output net only. The register substitution consists of the register string and the register index format. Example:

Gate uses: ccu_fetch_reg_3_

Specify the rule as: %s_reg_%d_

Causes the gate "ccu_fetch_reg_3_" signal to be mapped to signal "ccu_fetch[3]" in the RTL.

Note:

A rule in the **Register** or **Tri-state** field can only contain one '%s' and it must be at the beginning of the rule. For example, the rule "%s_reg_%d" is valid but the rule "%s_reg_%d" is not.

Applying Rules

The rules are applied sequentially in the order listed in the mapping rule table (both regular expression and predefined rules are applied).

Example 1:

Assuming the **Regular Expression** rules are:

a => x

b => y

`c => z`

and the **Predefined** rules are:

Hierarchical Separator: `#`

Register: `%s_reg_%d_`

This causes the register output net “top.m_a.m_b#m_c#test_reg_3_” to be mapped to “top.m_x.m_y.m_z.test[3]”.

Example 2:

Assuming the **Regular Expression** rules are:

`a => x`

`ab => xy`

This causes the second rule to become useless because the `a=>x` rule is applied first (all ‘a’ is replaced by ‘x’, so that there is no ‘ab’ strings).

Example 3:

Assuming the **Register** field of the **Predefined** rules include:

`%s_Z_%d`

and a library cell for a register is defined as:

`FF abc_Z_6(.CK(clk), .Q(n123), .QN(n124), .RN(reset));`

This causes the library cell `abc_Z_6` to be mapped to the RTL register signal `abc[6]`. If either the simulation library or the symbol library is available for the flip-flop (FF), the register output nets, `n123` and `n124`, are also mapped to `abc[6]` and its inversion respectively. In addition, the `clk` and `reset` signals are mapped according to the attributes of the CK and RN pins they are connected to and the syntax of the RTL code that defines register `abc[6]`.

Note:

It is very common for an RTL register net to be synthesized to a gate-level flip-flop or latch library cell; therefore, the recommendation is to first correlate a register library cell to its corresponding register net using the **Regular Expression** and/or **Predefined** rules. Then the correlation algorithm correlates the nets connected to the cell according to the pin attributes of the cell from either the symbol library or the simulation library.

5

Appendix B: Design Extraction Messages Overview

The design extraction engine, which is called CG2, checks target scope and exclude module before code generation.

1. Target Scope Check

If target scope is not specified correctly, `Add/Set scope failed` messages are reported. CG2 does not perform extraction. Review the target scope setting. For example,

[CG2 Log]: Target scope is not specified. Set scope failed.

2. Exclude Module Check

If exclude module is not specified correctly, `Failed to add exclude module` messages are reported. Review the exclude module setting. For example,

[CG2 Log]: Fail to add exclude module, module "test" not found.

After the checks, CG2 starts to collect viewing scope, traverse and process the design.

If there are VHDL scopes under the target scope, it is not supported currently. The following message is reported.

[CG2 Log]: VHDL is not supported yet.

When traversing/processing the design, CG2 skips statements in the initial block and protected module/interface/package with [CG2 Info] messages. If there are some problems or unsupported constructs in the design, [CG2 Warning] and [CG2 Error] messages are reported. Note that the View, File and Line information are also reported to help users check their design. For example, the following message means that class variable "packet_obj" is not supported, which can be found inside module "CPU" on line 26 in "test.v".

[CG2 Warning]: Class variable is not supported
Verbose: class variable is "packet_obj"

View: "CPU"

File: "test.v"

Line: "26 - 26"

Messages with severity Info:

```
[CG2_Info]: Skip protected module/interface/package  
[CG2_Info]: Skip statements in the initial block
```

Messages with severity Warning:

```
[CG2_Warning]: Get object of external reference failed  
[CG2_Warning]: Get definition of instance failed  
[CG2_Warning]: Fail to get signal handle by name  
[CG2_Warning]: Fail to process bind statement  
[CG2_Warning]: Traverse loop generate statement failed  
[CG2_Warning]: System function call $random found  
[CG2_Warning]: Load design error existed in interface  
[CG2_Warning]: Load design error existed in module  
[CG2_Warning]: Load design error existed in package  
[CG2_Warning]: VHDL entity will be translated into Verilog module  
[CG2_Warning]: VHDL instance will be translated into Verilog instance  
[CG2_Warning]: Class type will be replaced to logic  
[CG2_Warning]: Class variable is not supported  
[CG2_Warning]: SVTB statement is not supported  
[CG2_Warning]: Clocking block is not supported  
[CG2_Warning]: Covergroup is not supported
```

Messages with severity Error:

```
[CG2_Error]: Expression count of the operator is not correct  
[CG2_Error]: Invalid index value specified  
[CG2_Error]: Program is not supported  
[CG2_Error]: Symbol library module is not supported  
[CG2_Error]: Fail to get return type of the function  
[CG2_Error]: Fail to find modport task/function definition
```